Reverse engineering firmware

Philips 50PUS7608/12 • Titan OS • chassis TPN236E

Wersja softu: **TPN236E_V236.006.169.001** | analiza pliku **upgrade.pkg** (236.006.135.001) | dokument: 2026-11-02

Cel: edukacyjny reverse engineering oprogramowania własnego telewizora oraz zbadanie, czy da się dodać prostą pozycję w menu wyświetlającą komunikat typu „Hello World”. Niniejszy dokument podsumowuje wszystko, co udało się ustalić **wyłącznie analizą offline** (bez dotykania telewizora), oraz opisuje, co można zrobić później — dysponując pendrive’em albo lutownicą i adapterem UART.

W skrócie (TL;DR)

- Plik dystrybucyjny **Autorun.upg** jest **zaszyfrowany AES** (entropia 8.0 w całości) — offline bezużyteczny.
- Plik **upgrade.pkg** (tzw. upgrade_loader) **NIE jest zaszyfrowany** — ma jawny spis partycji i daje się w pełni rozpakować.
- Warstwy partycji: `gzip → NFSB (podpis Novateka) → squashfs 4.0 (LZO)`. Wszystko otwarte 7-Zipem.
- Platforma: SoC **Novatek NT72xxx**, **ARM 32-bit (armhf)**, Linux **Buildroot + BusyBox 1.35**.
- UI/menu Philipsa to **aplikacja webowa** (HTML/JS/CSS, webpack) — to najłatwiejszy cel dla „Hello World”.
- Konto **root nie ma hasła**; na porcie szeregowym działa **getty** → UART daje natychmiastowy root shell.
- Jest mechanizm **autostartu z USB** i hook `run_custom.sh` — droga do uruchomienia kodu bez łamania podpisu.
- Ściana: wgranie **zmodyfikowanego obrazu** normalnym update’em wymaga podpisu NFSB + secure boot — tego offline nie obejdziesz.

1. Platforma sprzętowa i systemowa

Ścieżki urządzeń w pakiecie (`/dev/nvt-part-name/...`) jednoznacznie wskazują SoC **Novatek** (rodzina NT72xxx, typowa dla telewizorów Philips/TPV na Titan OS). Rootfs to zwykły osadzony Linux zbudowany Buildrootem.

Element	Ustalenie
SoC	Novatek NT72xxx
Architektura	ARM 32-bit, armhf (ld-linux-armhf.so.3)
System	Linux, Buildroot, BusyBox 1.35.0, glibc 2.4
Init	BusyBox init + /etc/inittab + /etc/init.d/Sxx
UI (menu)	Aplikacja webowa TPV (HTML/JS/CSS) renderowana w stosie Wayland; obecny też silnik Cobalt (YouTube)
Zabezpieczenia	SELinux (targeted), PAM, audyt; NFSB-podpis obrazów; secure boot (uboot/secos)

2. Format firmware i tablica partycji

Dwa pliki to dwa różne formaty kontenera `TIMG`:

- **Autorun.upg** — nagłówek `TIMGPKGVER40`, od razu dane o entropii 8.0 → **szyfrowanie AES**, brak czytelnej struktury.

- **upgrade.pkg** — nagłówek `TIMG`, a od offsetu 0x120 jawna tablica **41 rekordów PIMG**: nazwa, urządzenie docelowe, typ kompresji, rozmiar.

Najważniejsze partycje z `upgrade.pkg`:

PIMG	Cel / urządzenie	Kompresja	Zawartość
`ker0/1`	`/dev/.../ker0`	none (uImage)	jądro Linux (~19 MB)
`fs0/1`	`/dev/.../fs0`	gzip	rootfs — squashfs LZO (~8 MB)
`ap0/1`	`/dev/.../apexe0`	gzip	aplikacja = UI/menu (~296 MB) — squashfs LZO
`apext0/1`	`/dev/.../apexe_ext0`	gzip	rozszerzenie aplikacji (~284 MB)
`secos0/1`	`/dev/.../secos0`	none (uImage)	Secure OS / TEE (~4 MB)
`uboot/xboot`	`/dev/.../boot0`	none	bootloadery + vtbl
`fdt0/1`	`/dev/.../fdt0`	none (uImage)	device tree
`demo`	`/dev/.../demo`	gzip	materiały demo (~190 MB)
`pq/aq/tcon/frc_fw`	`/dev/.../pq ...`	gzip/none	tablice obrazu, panel, FRC

3. Warstwy obrazu i jak to rozpakować

Każda partycja aplikacyjna (fs0, ap0) ma trzy warstwy. Procedura, którą przeszliśmy offline:

- 1. Z `upgrade.pkg` wyciąć strumień **gzip** danego rekordu PIMG i rozpakować → obraz `NFSB`.
- 2. `NFSB` to podpisany wrapper Novateka: nagłówek 0x800 B + dwa skróty SHA-256. Właściwy system plików zaczyna się od offsetu **0x1000**.
- 3. Od 0x1000 jest **squashfs 4.0 (LZO)** — wylistować/wypakować 7-Zipem.

Użyte narzędzia (Windows): `7-Zip 26` (C:\Program Files\7-Zip\7z.exe) obsługuje squashfs; Python 3 do cięcia i gzip. Skrypty pomocnicze: `scan_fw.py`, `parse_pimg.py`, `extract_pimg.py`. Przykładowe komendy:

```
# wylistowanie partycji (TOC)
python parse_pimg.py upgrade.pkg

# wypakowanie rootfs i aplikacji do obrazow NFSB
python extract_pimg.py upgrade.pkg extracted fs0 ap0

# wyciecie squashfs (NFSB zaczyna fs od 0x1000) i listing
"C:\Program Files\7-Zip\7z.exe" l ap0.squashfs
```

Część A — architektura UI i jak dodać „Hello World” do menu

Interfejs (launcher, menu, ustawienia, kreatory) to **aplikacja webowa TPV** spakowana webpackiem, leżąca w partycji apexe pod ścieżką:

```
/mtd_apexe/board/mframework/webui/PHI/page/
    index.html      <- bootstrap, definicje okien
    main.js         <- webpack bundle
    main.css (637 KB)
    lib/            <- win-manager.js, win-config.js, i18n/<LANG>.js, 186 x .js
    pages/          <- 34 kategorie stron (system, quickmenu, options, main, ...)
    icons/ assets/  <- grafika (PNG/SVG)
```

W `index.html` rejestrowana jest mapa wszystkich stron/menu — obiekty `window.Pages` oraz `window.PathToWin`. Każda trasa wskazuje na moduł w `pages/`. Przykłady istniejących tras:

```
window.Pages = {
  "quickmenu": "/quickmenu/quickmenu",
  "opt":       "/options/opt",
  "launcher":  "/main/launcher",
  "factory":   "/system/factory",
  "debugmenu": "/system/debugmenu",
  "all":       "/settings_RDL/all",  ...
};
```

UI rozmawia z natywnym backendem przez funkcje `Rpc...()` (np. `RpcSystemRegionGet`, `RpcFactoryGetCertApp`) — to warstwa JSON-RPC do procesu natywnego. Sama pozycja menu i jej widok to jednak czysty HTML/JS.

Plan dodania pozycji „Hello World” (koncepcyjnie)

- Sklonować istniejącą prostą stronę z `pages/` (np. z `quickmenu` lub `options`) jako nowy widok wyświetlający napis.
- Zarejestrować trasę w `window.Pages` i `window.PathToWin` (warstwa okna).
- Dodać klucz tłumaczenia w `lib/i18n/ENG.js` (i ewentualnie innych języków).
- Podpiąć otwarcie nowej strony z istniejącego menu (np. pozycja w szybkim menu).

Co można zrobić bez TV już teraz: całą tę warstwę da się **prototypować w przeglądarce na PC** — wypakowane `page/` otworzyć lokalnie, a brakujące wywołania `Rpc...` zaślepić (stuby zwracające `Promise.resolve(...)`). Pozwala to zbudować i obejrzeć widok „Hello World” zanim w ogóle dotkniesz telewizora. **Uwaga:** apexe jest w squashfs podpisanym NFSB — modyfikacja „w miejscu” i tak wymaga drogi z Części B, żeby zmiana ruszyła na sprzęcie.

Część B — rozruch systemu i drogi dostępu

Analiza rootfs (`fs0`) pokazała kilka realnych punktów zaczepienia. Wszystkie ustalone offline.

B1. Konto root bez hasła

Plik `/etc/shadow` — drugie pole (hash hasła) dla roota jest puste:

```
root::::::::
daemon:*:::::::
...
```

Puste pole = brak hasła. W połączeniu z konsolą szeregową daje to shell roota bez logowania.

B2. Konsola szeregowa (UART)

`/etc/inittab` stawia proces logowania na porcie szeregowym ttyS0:

```
ttyS0::respawn:/sbin/getty -L  ttyS0 0 vt100   # GENERIC_SERIAL
```

Po podłączeniu adaptera USB↔UART (3,3 V) do pinów debugowych na płycie głównej dostajesz prompt — i, przy pustym haśle roota, **powłokę roota**. To najpewniejsza droga do żywego systemu.

B3. Autostart z USB

Skrypt `/etc/init.d/S21idtv.sh` przy starcie wykonuje pierwszy istniejący `run.sh` z listy, która zawiera także pendrive:

```
run_path="/mtd_apexe/board/run.sh:/media/sda1/board/run.sh:/media/sda/board/run.sh"
for run_p in $run_path; do
    if [ -e ${run_p} ]; then cd `dirname ${run_p}`; source ./run.sh; break; fi
done
```

Warunkowane zmienną `bsp_only` z device tree (`/sys/firmware/devicetree/base/nvt_debugger/bsp_only`). Normalnie wygrywa wewnętrzny apexe, ale to pokazuje, że istnieje ścieżka ładowania skryptu z USB — kod wykonywany w locie, poza podpisanym obrazem.

B4. Hook run_custom.sh

Główny `board/run.sh` aplikacji sam dociąga skrypt rozszerzenia, jeśli istnieje:

```
if [ ! -f "./run_custom.sh" ]; then echo "./run_custom.sh not found"; return; fi
source ./run_custom.sh
run_custom_main
```

B5. Zapisywalne, niepodpisane partycje

Skrypty startowe operują na partycjach `/mtd_rwarea` i `/mtd_apdat` (np. `/mtd_rwarea/sw_version.txt`, konfiguracje, symlinki). Są one **zapisywalne i nie są objęte podpisem NFSB** — to naturalne miejsce na trwałe, własne pliki/ustawienia, których nie trzeba podpisywać.

4. Ściana: czego NIE da się obejść offline

- Obrazy partycji są opakowane w **NFSB** z dwoma skrótami SHA-256 i (najpewniej) podpisem — normalny update zweryfikuje je przed zapisem.
- Bootloader (uboot/xboot) + **secos (TEE)** realizują **secure boot** — klucze są wtopione w SoC Novatek.
- **Autorun.upg jest zaszyfrowany AES** — bez klucza z urządzenia nie odszyfrujesz go offline.
- Wniosek: **czytanie i modyfikowanie offline jest w pełni możliwe**, ale **wgranie zmodyfikowanego obrazu** normalną drogą — nie. Dlatego własny kod uruchamiamy przez UART / USB / zapisywalne partycje, a nie przez podmianę squashfs.

5. Co możesz zrobić później

Ścieżka 1 — tylko pendrive (bez otwierania TV)

- Przygotować pendrive z katalogiem `/board/run.sh` (prosty skrypt, np. log/echo) i sprawdzić, czy zostanie wykonany przy starcie.
- Zbadać dostępne w menu tryby serwisowe/debug (strony `/system/debugmenu`, `/system/factory`) — część z nich otwierają kody na pilocie.
- Zweryfikować warunek `bsp_only` i zachowanie pętli autostartu (czy da się zadziałać bez wewnętrznego apexe).
- Niskie ryzyko: nic nie lutujesz, nie ruszasz podpisów; najwyżej skrypt się nie uruchomi.

Ścieżka 2 — lutownica + adapter USB↔UART (root shell)

- Zlokalizować na płycie piny **UART** (RX/TX/GND) — zwykle 3 pola lutownicze; poziom **3,3 V**. Nie podłączać linii zasilania adaptera do TV.
- Podłączyć adapter (np. CP2102/FTDI 3,3 V), ustawić terminal (często 115200 8N1) i obserwować log U-Boot/kernela przy starcie.
- Po starcie pojawi się `getty` → zalogować się jako **root** (puste hasło).
- Z roota: dowolne eksperymenty na żywym systemie, w tym pierwsze „Hello World” (np. na konsoli, w logu, albo wstrzyknięte do UI).
- Ryzyko: utrata gwarancji, możliwość uszkodzenia przy zwarciu/złym napięciu. Pracować ostrożnie, najlepiej z izolacją.

Ścieżka 3 — „Hello World” w menu (docelowo)

- Krok 1 (PC): zbudować widok „Hello World” w wypakowanym `page/` i obejrzeć w przeglądarce ze stubami `Rpc...`.
- Krok 2 (TV): mając root (UART), uruchomić UI z **zapisywalnej** kopii `page/` (overlay/bind-mount w `/mtd_rwarea`) zamiast z podpisanego squashfs.
- Krok 3: podpiąć start tej kopii przez `run_custom.sh` / autostart z USB, żeby zmiana była trwała bez ruszania podpisu NFSB.

6. Mapa łańcucha weryfikacji (update loader)

Aby ocenić, czy **update loaderem** da się wgrać zmodyfikowany obraz, wyciągnięto z `upgrade.pkg` komponenty rozruchu (`uboot`, `secos`, `fdt`, `eburner`), rozpakowano nagłówki uImage i przeskanowano zawartość.

Format NFSB — co dokładnie zabezpiecza obraz

Nagłówek `NFSB` (0x800 B) zawiera: etykietę `sha256`, dwa skróty **SHA-256** zapisane jako ASCII (digest nagłówka i digest payloadu), a następnie **256-bajtowy blok binarny** (offset 0xb0–0x1b0) i wypełnienie zerami; właściwy squashfs zaczyna się od 0x1000.

- 256 bajtów = **podpis RSA-2048**. Oba obrazy (fs0, ap0) mają identyczny układ, ale **różne podpisy i różne hasze** — to podpis per-obraz, nie suma kontrolna.
- Można przeliczyć SHA-256 po modyfikacji, ale **nie da się sfałszować podpisu RSA** bez klucza prywatnego producenta.

Komponenty rozruchu — stan i znaleziska

Komponent	Co to	Stan	Istotne znaleziska
U-Boot 2016.07 (n220x)	pierwotny bootloader	**ciało ZASZYFROWANE**	brak jawnych stringów poza nazwą uImage → zamknięty łańcuch
secos	Secure OS / TEE	**ZASZYFROWANE**	obsługa OTP; wysoka entropia
fdt	device tree	**SPAKOWANE**	węzeł nvt_debugger/bsp_only czytelny tylko w runtime (/sys)
eburner	środowisko Linux (kernel+initramfs) do wypalania eMMC	**JAWNE**	RSA (Trsa_), SHA256 (ARMv8), PKCS7/X.509 public_key_verify_signature, OTP (OTPDa/OTPV), sb_enable

Łańcuch zaufania (secure boot)

```
BootROM  (w SoC, klucz zakotwiczony w OTP / eFuse)
   +-> xboot / U-Boot 2016.07     [cialo zaszyfrowane]
          +-> secos (TEE)          [zaszyfrowany]
                 +-> kernel / eburner
                        +-> weryfikacja NFSB (RSA-2048 + SHA-256)
                               +-> zapis partycji TYLKO po poprawnym podpisie
```

bsp_only / nvt_debugger

Flaga `bsp_only` z węzła device tree `nvt_debugger` (czytana w runtime przez `S21idtv.sh`) decyduje tylko o tym, czy przy starcie auto-uruchomić apexe — a więc czy **ścieżka USB** `/media/sda1/board/run.sh` staje się osiągalna. **Nie** wpływa na egzekwowanie podpisu obrazu.

Werdykt: wgranie zmodyfikowanego obrazu loaderem

- **Nie.** Pełny stos weryfikacji (RSA-2048 + SHA-256 + OTP + sb_enable) jest w `eburner`, a `uboot`/`secos` są zaszyfrowane — **łańcuch jest zamknięty**.
- Brak możliwego do audytu **offline** obejścia: nie da się przejrzeć uboota/secos pod kątem wyłączonej weryfikacji, bo ich ciała są zaszyfrowane.
- Hooki inżynieryjne (`bsp_only`, dual-bank, `eburner`) dotyczą **uruchamiania aplikacji / wypalania**, nie **egzekwowania podpisu**.
- Realna droga dla naszej modyfikacji pozostaje bez loadera: **root przez UART → overlay na /mtd_rwarea → run_custom.sh / USB** (nie wymaga podpisu).

Pliki wyciągnięte do analizy: `extracted/components/{uboot,secos0,fdt0,eburner}.{bin,payload}`.

6.1. Wnętrze flashera — nvt_eburner

`eburner` okazał się pełnym środowiskiem: kernel ARM64 z osadzonym **initramfs** (strumień xz → cpio, ~9,7 MB). Właściwy flasher to ELF **ARM 32-bit** `/bin/nvt_eburner` (774 KB), sterowany skryptem `nvt_update.sh`.

- Przebieg: z `/proc/cmdline` czytane jest `burn_file_name=*.pkg`, plik brany z USB (`/media/sda1`), uruchamiany `nvt_eburner`, potem `reboot`.
- Istnieje też sieciowy tryb **FW_FLOOD_MODE** (`nvt_fw_flood_receiver` + avahi) — wgrywanie przez Ethernet, gdy brak pliku na USB.
- Funkcje w binarce: `nvt_eburner_verify_whole_image`, `nvt_ebruner_md5_part_verify`, `gen_md5tbl_in_ram`, `get_per_img_by_name`, `write_boot_flag`; komunikaty verify success!/failed!, %s invalid header sign !.

6.2. Klucz weryfikujący — res/keys

W initramfs flashera leży plik `res/keys` (1387 B) — **klucz publiczny RSA-2048** w tekstowym formacie osadzonego weryfikatora (styl vboot/avb):

```
{ 64, 0x<n0inv>, { ...64 slowa 32-bit = modulus N... },
                 { ...64 slowa 32-bit = R^2 mod N... } }
```

- 64 słowa 32-bit = **2048 bitów**; `n0inv` = stała Montgomery (−1/N mod 2^32); drugi blok = R^2 mod N; wykładnik domyślnie 65537.
- Czyli `nvt_eburner` weryfikuje podpis RSA-2048 z `NFSB` tym kluczem **przed zapisem** partycji.
- Warstwa **MD5** (`gen_md5tbl_in_ram`) to integralność transferu, **nie** granica bezpieczeństwa.

6.3. Czy da się podmienić klucz albo pominąć weryfikację?

- **Brak flagi pomijającej weryfikację** — tokeny `skip/force/disable` w binarce dotyczą obsługi błędów i `force_ro` eMMC, nie podpisu.
- Klucz jest plikiem w initramfs, ale `eburner` sam jest **podpisaną partycją PIMG** ładowaną i weryfikowaną przez `uboot/secos` zakotwiczone w OTP.
- Żeby wstawić własny klucz do `res/keys`, trzeba przepakować i **podpisać** eburner — czego nie da się zrobić. **Zamknięte koło.**
- Werdykt bez zmian: modyfikacja odrzucona; przeliczenie haszy nie pomaga (podpisu RSA nie podrobisz, klucza nie podmienisz).

6.4. Wbudowany overlay apexe — droga wdrożenia bez podpisu

Skrypt `profile_emmc` montuje partycję aplikacji na dwa sposoby, zależnie od flagi device tree `nvt_debugger/ap_overlay`:

```
AP_OVERLAY=$(cat /sys/.../nvt_debugger/ap_overlay)
if [ -z "$AP_OVERLAY" ] || [ $AP_OVERLAY = 0 ]; then
    func_mount_fs_secure  apexe   # produkcja: mount -t nfsb (RO, podpis sprawdzany)
else
    func_mount_overlayfs  apexe   # overlay: lower=squashfs(RO) + upper=apdat(RW)
fi
```

- Telewizor ma **wbudowany overlayfs nad apexe**: lower = podpisany squashfs, **upper = zapisywalna, niepodpisana partycja apdat**, scalony widok w `/mtd_apexe`.

- Pliki w `/mtd_rwarea/ap_upper/...` **nadpisują** squashfs **bez podpisu** — to zamierzony punkt rozszerzeń.
- Odkrycie poboczne: `nfsb` to także **typ montowania w kernelu** (weryfikacja podpisu przy montowaniu), nie tylko format nagłówka.
- Dwie drogi wdrożenia naszego „Hello World”: **(A)** tryb overlay (gdy `ap_overlay≠0`) — pliki + `run_custom.sh` do upper; **(B)** produkcja — z roota (UART) **bind-mount** katalogu `page` z `/mtd_rwarea`.
- Gotowe skrypty: `helloworld_update/deploy/` (`deploy_from_root.sh`, `hw_overlay.sh`, `run_custom.sh`, `inject-helloworld.js`).
- Integracja z UI: zarządca okien jest globalny jako `window.S`, a strony rejestrują klawisze przez `S.onkeydown(fn)`. `inject-helloworld.js` wpina tak **niebieski przycisk (BLUE)** — otwiera ekran „Hello World” (zweryfikowane w przeglądarce ze stubem `window.S`).
- Natywny **kafelek w Szybkim menu**: pozycje mają akcje wzorca `S.open(F.help)`. `patch_native_tile.sh` dokłada pozycję do tablicy + `case` do switcha + klucz i18n (podmiana literalna na kopii, kotwice unikalne, idempotentna, odwracalna; poprawność potwierdzona `node --check`).

6.5. Oficjalny update, typy buildów i recovery (dokumentacja Titan OS)

Oficjalna dokumentacja Titan OS (firmware installation guide) potwierdza i uzupełnia nasze ustalenia:

- **Typy buildów:** wersje kończące się na `1` = produkcyjne, na `0` = **debug** („extra logs and diagnostic tools”). Nasz soft …`169.001` i analizowany …`135.001` to **produkcja**.
- **Trop:** build **debug** (…`0`) ma diagnostykę i może mieć poluzowane flagi `nvt_debugger` (`bsp_only` / `ap_overlay`), które u nas bramkują wbudowany overlay i autostart z USB — potencjalnie łatwiejszy dostęp **bez** łamania podpisu. Oficjalnie dostępny tylko dla partnerów.
- **pkg.burn.7z + „empty burning”:** udokumentowana procedura USB do przełączania produkcja↔debug (trzymanie joysticka power ~10–15 s przy podłączaniu zasilania). Instaluje **podpisany** firmware producenta — nie obchodzi podpisu.
- **Auto-update z USB** (`upgrade.pkg` w rootcie → replug) — dokładnie potwierdza to, co odtworzyliśmy z `nvt_update.sh` (`burn_file_name=*.pkg`, `/media/sda1`).
- **Recovery:** Settings → General Settings → **Reinstall** — miękki reset po eksperymentach.
- Dokument **nie** opisuje podpisywania, bootloadera, serial ani menu serwisowego — nie zaprzecza więc głębszym ustaleniom (RSA-2048, secure boot, NFSB).

Analizowane pliki (upgrade.pkg + loader) pochodzą z archiwum społecznościowego toengel.net/philipsblog; build debug dla TPN236E nie był dostępny.

7. Pliki i narzędzia w projekcie

```
C:\App Projects\philipsTv\
  Autorun.upg                          # dystrybucyjny, AES (slepa uliczka)
  upgrade.pkg                          # upgrade_loader, NIEszyfrowany (zrodlo)
  TPN236E_*.zip                        # opakowania powyzszych
  extracted\
    fs0.img / fs0.squashfs             # rootfs (NFSB + squashfs)
    rootfs_fs0\                         # wypakowany rootfs
    ap0.img / ap0.squashfs             # aplikacja (apexe)
    apexe\board\mframework\webui\PHI\page\   # web UI (menu)
  (skrypty analityczne w katalogu scratchpad sesji)
```

Dodatek A — UART krok po kroku (bezpieczne podłączenie)

Rozszerzenie Ścieżki 2. Celem jest **root shell przez konsolę szeregową** (w tym firmware potwierdzone: `getty` na `ttyS0` + konto root bez hasła). Dokładne rozmieszczenie pól UART różni się między rewizjami płyty i nie da się go ustalić z samego firmware — poniżej jest uniwersalna procedura ich znalezienia i bezpiecznego podłączenia.

A.1. Czego potrzebujesz

- Adapter **USB–UART** na logice **3,3 V** (CP2102 / CH340 / FTDI). Najlepiej z obsługą także **1,8 V** lub osobny konwerter poziomów.
- 3 przewody (żeńsko-żeńskie lub z chwytakami); ewentualnie lutownica i cienki drut do pól lutowniczych.
- Terminal szeregowy (PuTTY / minicom / screen), **multimetr**, opcjonalnie analizator logiczny.

A.2. Bezpieczeństwo — najpierw

- Telewizor zasilany z sieci. Montaż adaptera rób przy **odłączonym zasilaniu**; log rozruchu czytasz dopiero po włączeniu — wtedy ostrożnie.
- Strona **zasilacza (PSU)** bywa pod napięciem sieci — nie dotykaj. Pracuj tylko po stronie cyfrowej (SoC/eMMC).
- **Wspólna masa obowiązkowa**; **nie podłączaj VCC** adaptera do płyty TV.
- Rozważ zasilacz izolowany; unikaj jednoczesnego dotykania metalu PC i płyty telewizora.

A.3. Znalezienie nagłówka UART

Szukaj grupy **3–4 pól/pinów w jednej linii**, zwykle blisko SoC lub kości eMMC, czasem jako nieobsadzony 4-pin. Jeśli jest sitodruk (`RX / TX / GND / 3V3`) — masz gotowe. Bez opisów identyfikuj multimetrem (A.4).

A.4. Identyfikacja pinów bez opisu (multimetr)

Pin	Jak rozpoznać
GND	Ciągłość do masy — ekran USB, radiator, duży plan masy (tryb ciągłości multimetru).
VCC 3,3 V	Stałe ~3,3 V DC względem GND po zasileniu. NIE podłączać do adaptera.
TX (z TV)	Spoczynkowo na poziomie logiki (3,3 lub 1,8 V); „drga” przy starcie, gdy wypisuje log. Na analizatorze: pakiety tuż po włączeniu.
RX (do TV)	Zwykle statyczne, często słaby pull-up; ustalane przez eliminację.

WAŻNE — poziom logiki. Zmierz napięcie spoczynkowe TX/VCC. Jeśli **~1,8 V**, UART działa na 1,8 V — użyj adaptera/konwertera 1,8 V. Podpięcie 3,3 V do 1,8 V grozi uszkodzeniem SoC.

A.5. Połączenie (krzyżujemy TX/RX)

```
TV GND  <->  adapter GND
TV TX   <->  adapter RX
TV RX   <->  adapter TX
TV VCC   ->  NIE podlaczac
```

A.6. Terminal i logowanie

- Port adaptera, **115200 8N1**, sterowanie przepływem: brak.
- Włącz TV i obserwuj log (U-Boot → kernel). Naciśnij **Enter** — pojawi się prompt `getty`.
- Zaloguj jako **root** (puste hasło). Sprawdź: `id`, `mount`, `ls /mtd_apexe`.
- Mając root → uruchom `deploy_from_root.sh` i resztę (overlay, inject, kafelek).

A.7. Rozwiązywanie problemów

- „Krzaki” na ekranie → zła prędkość: spróbuj 115200, potem 57600 / 38400 / 921600.
- Nic nie widać → zamień **RX↔TX**, sprawdź wspólną **masę**.
- Dobry baud, a nadal śmieci → poziom logiki to pewnie **1,8 V**, nie 3,3 V (zmierz i użyj właściwego).
- Log leci, ale brak promptu → konsola może być ograniczona; sam log rozruchu i tak bywa cenny diagnostycznie.

Rozmieszczenie pól różni się między rewizjami płyty — zidentyfikuj je oględzinami i pomiarem przed podłączeniem.

Zastrzeżenie: dokument dotyczy analizy własnego urządzenia w celach edukacyjnych. Ingerencja sprzętowa (UART, otwieranie obudowy) oznacza utratę gwarancji i ryzyko trwałego uszkodzenia. Modyfikacje wykonujesz na własną odpowiedzialność.